

Phase-1: Beginner's Step-by-Step Guide — 'Google of Finance' MVP (Free Tools)

Generated: 2025-10-29 10:04 UTC

This guide walks you step-by-step to build a free, student-friendly MVP for a lightweight 'Google of Finance' website. You will use free tools only: Python, Streamlit, yfinance, Hugging Face models (FinBERT), Google Colab (for training if needed), and free hosting via Streamlit Cloud / Vercel. The guide focuses on Phase-1: a working demo that accepts a company name, fetches data live, runs simple models, and shows results — without storing user data.

Contents

1. Prerequisites
 2. Project structure
 3. Local setup (Python + virtualenv)
 4. Streamlit app skeleton (full code)
 5. Fetching stock & financial data with yfinance
 6. Simple LSTM forecaster (concept + Colab snippet)
 7. Sentiment analysis with FinBERT (headlines scraping)
 8. Combining outputs and rule-based summary
 9. Deploying to Streamlit Cloud (free)
 10. Testing, debugging, and next steps
- Appendix: Helpful commands & resources

1. Prerequisites

Hardware: A laptop (Windows/Mac/Linux). A modern browser (Chrome/Firefox). Internet access. Software: Python 3.10+ (or 3.8+), Git (optional but recommended), Google account for Colab/Streamlit Cloud. Accounts & Free services: GitHub (free), Streamlit Cloud (free tier), Hugging Face account (recommended), Google Colab (free). Python packages used: streamlit, yfinance, pandas, numpy, matplotlib, transformers, torch (or tensorflow), requests, beautifulsoup4

2. Project structure

A simple folder layout to begin: project/ `app.py` # Streamlit app (main) `requirements.txt` `modules/` `data_fetch.py` # wrappers for yfinance & news scraping `forecast.py` # LSTM training & predict (or placeholder) `sentiment.py` # FinBERT sentiment wrappers `README.md`

3. Local setup (Python + virtualenv)

1. Install Python (if not installed). Verify: `python --version` 2. Create and activate a virtual environment: `python -m venv venv` # Windows: `venv\Scripts\activate` # macOS / Linux: `source venv/bin/activate` 3. Create `requirements.txt` with the following lines: `streamlit yfinance pandas numpy matplotlib requests beautifulsoup4 transformers torch scikit-learn` 4. Install packages: `pip install -r requirements.txt`

4. Streamlit app skeleton (full code)

Below is a complete Streamlit app skeleton combining the major steps. Save as `app.py` in project root.

```
# app.py - Streamlit MVP skeleton
import streamlit as st
import yfinance as yf
import pandas as pd
import matplotlib.pyplot as plt
from modules import data_fetch, sentiment, forecast

st.set_page_config(page_title="Google of Finance - Demo", layout="wide")
st.title("Google of Finance - Demo (Student MVP)")
query = st.text_input("Enter company name or ticker (e.g., TCS.NS or Tata Motors)")

if query:
    with st.spinner("Fetching data..."):
        # 1. Resolve ticker & fetch history
        ticker_symbol = data_fetch.resolve_ticker(query)
        if ticker_symbol is None:
            st.error("Could not find ticker for that query.")
        else:
            st.subheader(f"Results for {ticker_symbol}")
            hist = data_fetch.get_price_history(ticker_symbol, period="2y")
            st.write(hist.tail())

            # Plot price
            fig, ax = plt.subplots()
            ax.plot(hist['Close'])
            ax.set_title("Price history (Close)")
            st.pyplot(fig)

            # 2. Forecast (placeholder - simple naive forecast or model)
            forecast_df = forecast.predict_next_days(hist, days=30)
            st.subheader("Forecast (next 30 days)")
            st.line_chart(forecast_df['Close'])

            # 3. Sentiment
            headlines = data_fetch.get_news_headlines(query, max_headlines=20)
            sent = sentiment.analyze_headlines(headlines)
            st.subheader("Sentiment Summary")
            st.json(sent)

            # 4. Financials
            fin = data_fetch.get_financials(ticker_symbol)
            st.subheader("Financial snapshot (last 2 years)")
```

```
st.write(fin)
```

5. Fetching stock & financial data with yfinance

Use yfinance to resolve tickers and fetch history & financials. Example helper (modules/data_fetch.py):

```
# modules/data_fetch.py
import yfinance as yf
import pandas as pd
from bs4 import BeautifulSoup
import requests

def resolve_ticker(query):
    # Basic heuristic: if contains dot or is uppercase, assume ticker; else try search via yfinance
    try:
        t = yf.Ticker(query)
        info = t.info
        if 'shortName' in info:
            return query
    except Exception:
        pass
    # Try search via yfinance's Tickers (simple fallback could be improved)
    # For MVP, accept input as ticker from user.
    return query # keep simple for MVP

def get_price_history(ticker, period="2y"):
    t = yf.Ticker(ticker)
    hist = t.history(period=period)
    hist = hist.reset_index()
    return hist

def get_financials(ticker):
    t = yf.Ticker(ticker)
    # financials returns annual or quarterly; convert to readable format
    try:
        fin = t.financials
        return fin
    except Exception:
        return {"error": "Could not fetch financials"}

def get_news_headlines(query, max_headlines=20):
    # Simple approach: Use Google News RSS via requests+BeautifulSoup or NewsAPI if available
    url = f"https://news.google.com/search?q={query}"
    resp = requests.get(url, headers={"User-Agent": "Mozilla/5.0"})
    soup = BeautifulSoup(resp.text, "html.parser")
    elems = soup.select("a.DY5T1d") # may change; fragile but works as MVP
    headlines = [e.get_text() for e in elems[:max_headlines]]
    return headlines
```

6. Simple LSTM forecaster (concept + Colab snippet)

For a student MVP, you can use a lightweight LSTM or even a naive walk-forward average. Training in Google Colab is recommended (free GPU if available). Below is a compact Colab-friendly snippet using TensorFlow/Keras.

```
# forecast.py (simplified) - uses past Close prices to predict next n days (naive or small LSTM)
import numpy as np
import pandas as pd

def predict_next_days(hist_df, days=30):
    # Naive approach: use last 10-day average as forecast for each next day
    last_close = hist_df['Close'].values[-10:]
    avg = last_close.mean()
    future = [avg for _ in range(days)]
    dates = pd.date_range(start=hist_df['Date'].iloc[-1] + pd.Timedelta(days=1), periods=days)
    return pd.DataFrame({'Date': dates, 'Close': future})
```

7. Sentiment analysis with FinBERT (headlines scraping)

FinBERT from Hugging Face works well for finance text. For MVP, analyze headlines (not full articles) to keep API calls light. Install transformers and torch. Example wrapper (modules/sentiment.py):

```
# modules/sentiment.py
from transformers import AutoTokenizer, AutoModelForSequenceClassification
import torch
import numpy as np

tokenizer = AutoTokenizer.from_pretrained("ProsusAI/finbert")
model = AutoModelForSequenceClassification.from_pretrained("ProsusAI/finbert")

def analyze_headlines(headlines):
    results = []
    for h in headlines:
        inputs = tokenizer(h, truncation=True, padding=True, return_tensors="pt")
        with torch.no_grad():
            out = model(**inputs)
            scores = out.logits.numpy()[0]
            # Softmax to probabilities
            probs = np.exp(scores) / np.exp(scores).sum()
            labels = ["neutral", "positive", "negative"] # depends on model mapping
            pred = labels[probs.argmax()]
            results.append({'headline': h, 'label': pred, 'probs': probs.tolist()})
    # Aggregate simple summary
    counts = {'positive':0, 'neutral':0, 'negative':0}
    for r in results:
        counts[r['label']] += 1
    total = len(results) if results else 1
    summary = {k: round(v/total,2) for k,v in counts.items()}
    return {'summary': summary, 'details': results}
```

8. Combining outputs and rule-based summary

Create a short rule-based 'AI Opinion' that combines signals: - If sentiment positive > 0.6 AND revenue growing => 'Bullish signal (positive sentiment + growth)' - If sentiment negative > 0.6 AND revenue declining => 'Bearish signal' - Else => 'Mixed signals — dig deeper.' This avoids LLM token costs and is explainable for investors.

9. Deploying to Streamlit Cloud (free)

1. Push your project to GitHub (public repo is fine for MVP). 2. Sign in to Streamlit Cloud with your GitHub account. 3. Click 'New app' → select repository and branch → set 'app.py' as entrypoint. 4. Add secrets if any (avoid storing API keys; prefer free scrapers and public data). 5. Deploy — Streamlit will build the environment using your requirements.txt Notes: If your app uses heavy models (FinBERT), consider lazy-loading the model (load only when needed) to reduce memory/time on cold starts.

10. Testing, debugging, and next steps

Testing tips: - Use print/logging and Streamlit st.write to debug. - Test with known tickers (AAPL, MSFT, TCS.NS) to verify data. - If scraping fails, fallback to NewsAPI free tier (100 calls/day) or limit headlines. Next steps after demo: - Add caching for recent queries (short TTL) to improve speed. - Improve forecasting with a trained LSTM (use Colab to train and export weights). - Add rate-limiting and polite scraping (respect robots.txt). - Prepare a short demo video (1–2 minutes) to show to mentors or at hackathons.

Appendix: Helpful commands & resources

Helpful commands: - git init; git add .; git commit -m "initial"; git push origin main - streamlit run app.py # to run locally
Resources: - yfinance docs: <https://pypi.org/project/yfinance/> - Hugging Face FinBERT: <https://huggingface.co/ProsusAI/finbert> - Streamlit: <https://streamlit.io> - Google Colab: <https://colab.research.google.com>